



TEMA 6 – PERSONALIZACIÓN Y CONFIGURACIÓN AVANZADA



Objetivos del tema

Al finalizar este tema, los alumnos podrán:

- Entender la configuración por defecto de Tailwind
- Personalizar colores, fuentes y tamaños
- Extender y sobrescribir estilos
- Usar variables CSS en Tailwind 4
- Activar y configurar **Dark mode**

Tema por defecto (default theme)

- Tailwind trae un **tema por defecto** en `tailwind.config.js`
- Incluye:
 - Colores predefinidos (`blue-500`, `gray-200`, etc.)
 - Tipografía (`font-sans`, `font-serif`, `font-mono`)
 - Tamaños de spacing (`p-1` a `p-64`)
 - Breakpoints responsive



Este tema se puede **extender o sobrescribir** según necesidades

Ejercicio 1

1. Crea un `div` con fondo `bg-blue-500` y texto `text-white`
2. Cambia a diferentes colores (`bg-red-500`, `bg-green-500`)
3. Cambia tipografía (`font-serif`, `font-mono`)

Personalizar colores, fuentes y tamaños

- Edita `tailwind.config.js`:

```
export default = {  
  theme: {  
    extend: {  
      colors: {  
        brand: '#1c64f2',  
      },  
      fontFamily: {  
        sans: ['Inter',  
        'sans-serif'],  
      },  
      spacing: {  
        128: '32rem',  
      },  
    },  
  },  
}
```

`colors.brand` → color personalizado

`fontFamily.sans` → fuente personalizada

`spacing.128` → padding/margin grande

Ejercicio 1

1. Extiende `tailwind.config.js` para añadir:
 - Un color `brand` (`#FF5733`)
 - Una fuente `custom` (`Roboto, sans-serif`)
2. Crea un `div` usando:

```
<div class="bg-brand text-white font-custom p-8">  
  Proyecto personalizado  
</div>
```

- ♦ Observa los cambios en la interfaz

Extender vs sobrescribir

- **Extender:** agregas nuevos valores al tema sin eliminar los existentes (`extend`)
- **Sobrescribir:** reemplazas valores del tema por defecto (`theme`)

Ejemplo:

```
theme: {  
  colors: {  
    blue: '#123456', // sobrescribe el azul por defecto  
  },  
  extend: {  
    colors: {  
      brand: '#FF5733', // añade color sin eliminar el resto  
    },  
  },  
}
```

Ejercicio 3

1. Sobrescribe el color azul predeterminado
2. Añade un color `secondary` sin eliminar los demás
3. Crea dos `div` mostrando ambos colores

Uso de variables CSS en Tailwind 4

Tailwind 4 integra **variables CSS** para más flexibilidad

- CSS:

```
:root {  
  --color-primary: #1c64f2;  
}
```

- tailwind.config.js:

```
module.exports = {  
  theme: {  
    extend: {  
      colors: {  
        primary:  
          'var(--color-primary)',  
      },  
    },  
  },  
}
```

Ejercicio 4

1. Crea una variable `--color-primary` en CSS (`:root`)
2. Añádela al `tailwind.config.js` como color `primary`
3. Usa `bg-primary` en un `div`
4. Cambia el valor de la variable en `:root` y observa el cambio dinámico

Dark Mode en Tailwind 4

- Tailwind soporta **Dark Mode** nativamente
- Configuración en `tailwind.config.js`:

```
module.exports = {  
  
  darkMode: 'class', // o 'media'  
  
}
```

```
Añadir en el input.css:  
@custom-variant dark  
(&:where(.dark, .dark *));
```

- Uso en HTML:

Prefijo **dark**: aplica estilos en modo oscuro

```
<body class="dark">  
  <div class="bg-white dark:bg-gray-800 text-black dark:text-white p-6">  
    Contenido adaptable al modo oscuro  
  </div>  
</body>
```

Ejercicio 5

1. Activa `darkMode: 'class'` en `tailwind.config.js`
2. Crea un `div` con:
 - `bg-white dark:bg-gray-800`
 - `text-black dark:text-white`
3. Añade la clase `dark` al `body` y observa el cambio
4. Alterna la clase `dark` dinámicamente (ejemplo con botón)

Resumen

Tailwind tiene un **tema por defecto** con colores, fuentes y spacing

Se puede **extender** (añadir) o **sobrescribir** (reemplazar)

Tailwind 4 permite usar **variables CSS** para mayor flexibilidad

Dark Mode integrado con prefijo **dark:**